

Projeto II: Sistema de Gestão Bibliotecária

1. O Desafio da Equipe

O grupo foi contratado para desenvolver o **LibriTech** (*vocês podem substituir o nome por outro*), um sistema robusto para o gerenciamento de uma biblioteca universitária. Neste projeto, o **MySQL** não será apenas um depósito de dados; ele atuará como o "cérebro" que protege as regras de negócio e a segurança. O **Java** servirá como a interface amigável que interage com esse cérebro.

A equipe deverá aplicar **todo** o conteúdo visto na disciplina, garantindo que a segurança seja imposta pelo banco de dados e não apenas pela interface gráfica.

2. A Base: Estrutura do Banco de Dados (DDL)

Antes de escrever qualquer linha de Java, a equipe precisa garantir que o alicerce (o banco `db_libritech`) seja sólido. Um banco mal projetado fará com que o código Java quebre ou fique complexo demais.

Vocês devem criar as tabelas abaixo respeitando as chaves (PK e FK) e os tipos de dados adequados.

1. Tabela `usuarios` (A Entidade Pai)

Esta tabela armazenará todos os atores do sistema. Para suportar a herança no Java (Aluno, Funcionario, Gerente), usaremos uma coluna discriminadora (`tipo`).

- **`id_usuario` (INT, PK, AUTO_INCREMENT):** A identidade única e imutável.
- **`nome` (VARCHAR):** Nome completo.
- **`cpf` (CHAR(11), UNIQUE):** Deve ser chave única para impedir duplicidade de cadastro. Usem `CHAR` pois tem tamanho fixo.
- **`email` (VARCHAR, UNIQUE):** Essencial para comunicação e login.
- **`senha` (VARCHAR):** Nunca armazenem senhas curtas; pensem no tamanho do hash.
- **`tipo` (ENUM ou VARCHAR):** Valores esperados: 'ALUNO', 'GERENTE', 'BIBLIOTECARIO', 'ESTAGIARIO'.

2. Tabela `Enderecos` (Normalização - 1FN)

Seguindo a **1ª Forma Normal** (evitar grupos de repetição e colunas compostas), o endereço **não** deve ficar na tabela de usuários.

- **`id_endereco` (INT, PK, AUTO_INCREMENT).**
- **`logradouro`, `bairro`, `cidade`, `uf`:** Campos atômicos.
- **`id_usuario_fk` (INT, FK):** Chave Estrangeira que liga este endereço a um usuário específico. Lembrem-se da Integridade Referencial: não se pode cadastrar um endereço para um usuário que não existe.

3. Tabela `Livros` (O Acervo)

- `id_livro` (INT, PK, AUTO_INCREMENT).
- `titulo` (VARCHAR): Título da obra.
- `autor` (VARCHAR): Para simplificar, pode ser um campo texto, mas na 3FN ideal seria uma tabela separada. Para este projeto, aceitaremos na tabela livros.
- `isbn` (VARCHAR, UNIQUE): O código universal do livro.
- `preco_custo` (DECIMAL(10,2)): **Atenção:** Nunca usem `FLOAT` ou `DOUBLE` para dinheiro. O tipo `DECIMAL` garante a precisão dos centavos.
- `quantidade_estoque` (INT): Controlado pelas triggers de empréstimo/devolução.
- `status` (VARCHAR): Ex: 'DISPONIVEL', 'INDISPONIVEL'.

4. Tabela `Emprestimos` (A Tabela Associativa)

Esta tabela resolve o relacionamento "Muitos-para-Muitos" entre Usuários e Livros.

- `id_emprestimo` (INT, PK, AUTO_INCREMENT).
- `id_usuario_fk` (INT, FK): Quem pegou o livro.
- `id_livro_fk` (INT, FK): Qual livro foi pego.
- `data_saida` (DATETIME): Data e hora exata da retirada (Use `DEFAULT CURRENT_TIMESTAMP`).
- `data_prevista` (DATE): Quando o livro deve voltar (Calculado pelo Java ou Procedure).
- `data_devolucao` (DATETIME): Inicialmente `NULL`. Só é preenchida quando o livro é devolvido.

5. Tabela `Multas` (Financeiro)

- `id_multa` (INT, PK, AUTO_INCREMENT).
- `id_emprestimo_fk` (INT, FK): Vincula a multa a um empréstimo específico.
- `valor` (DECIMAL(10,2)): Valor calculado pela procedure de atraso.
- `pago` (BOOLEAN ou TINYINT): 0 para pendente, 1 para pago.

6. Tabela `Log_Auditoria` (Segurança)

Esta tabela será alimentada **exclusivamente** pela Trigger de Auditoria.

- `id_log` (INT, PK, AUTO_INCREMENT).
- `tabela_afetada` (VARCHAR): Ex: 'Livros'.
- `acao` (VARCHAR): Ex: 'DELETE', 'UPDATE PRECO'.
- `usuario_responsavel` (VARCHAR): Quem estava logado no banco (função `USER()` ou `CURRENT_USER()`).
- `dados_antigos` (TEXT): JSON ou texto contendo o registro `OLD` que foi apagado.
- `data_hora` (TIMESTAMP): `DEFAULT CURRENT_TIMESTAMP`.

3. Segurança no Banco (O "Coração" do Projeto)

A equipe deve implementar o princípio do **Privilégio Mínimo**. Vocês criarão 4 usuários distintos no MySQL, e a aplicação Java deverá conectar-se usando essas credenciais específicas, dependendo de quem está fazendo login.

Abaixo estão as regras que devem ser aplicadas via permissões (GRANT/REVOKE):

1. **usr_gerente (O Administrador):** Este usuário deve ter acesso total ao banco (ALL PRIVILEGES). A função dele é gerencial: ele é o único autorizado a ver dados financeiros sensíveis (tabela de Multas e Preços) e o único capaz de executar o comando de Backup, garantindo a recuperação de desastres.
2. **usr_bibliotecario (O Operador):** É o responsável pela rotina diária. Ele precisa de permissão para consultar livros, registrar novos empréstimos e atualizar devoluções. Porém, por segurança, ele **não** deve ter permissão para apagar registros da tabela de auditoria, garantindo a rastreabilidade do sistema.
3. **usr_estagiario (O Usuário Restrito):** Este perfil serve para provar a segurança do sistema. Ele deve ter permissões mínimas: apenas consultar o acervo e inserir empréstimos simples. A equipe deve **revogar explicitamente** o direito de DELETE deste usuário. Se ele tentar apagar um livro, o banco deve bloquear.
4. **usr_aluno (O Visitante):** Este usuário terá acesso apenas de leitura (SELECT). Para proteger dados sigilosos (como o preço de custo dos livros), ele **não** deve ter acesso direto às tabelas físicas. O acesso dele deve ser feito exclusivamente através de **Views** públicas criadas pela equipe.

4. Objetos Ativos

A equipe deve implementar os seguintes objetos no banco:

1. Quatro Stored Procedures (Procedimentos Armazenados)

1. **sp_transacao_emprestimo(id_user, id_livro):** Inicia Transação. Verifica se o usuário tem pendências. Verifica estoque. Insere na tabela Empréstimos. Decrementa 1 no estoque da tabela Livros. Se qualquer passo falhar (ex: estoque zero), faz ROLLBACK; senão, COMMIT.
2. **sp_renovar_emprestimo(id_emprestimo):** Verifica se o livro possui status de "Reservado" por outro usuário. Se estiver livre, atualiza a data_prevista somando 7 dias. Se houver reserva, nega a operação e retorna uma mensagem de erro. Deve garantir a consistência dos dados.
3. **sp_calcular_multa(id_emprestimo, OUT valor_multa):** Recebe o ID do empréstimo. Compara a data_prevista com a data atual (ou data de devolução). Se houver atraso, calcula o valor (ex: Dias * R\$ 2,00). Retorna esse valor no parâmetro de saída OUT para que o Java possa exibir na tela antes de confirmar o pagamento.
4. **sp_transacao_cadastro_completo(nome, cpf, ..., rua, num):** Inicia Transação. Insere os dados na tabela Usuarios. Recupera o ID gerado (LAST_INSERT_ID()). Insere os dados na tabela Enderecos usando esse ID. Se o

endereço falhar, desfaz a inserção do usuário (ROLLBACK) para não deixar registros órfãos.

2. Quatro Triggers (Gatilhos)

1. **trg_trava_horario_comercial**: Dispara BEFORE INSERT/UPDATE. Se for fora do horário (08h-18h), aborta com erro (SIGNAL SQLSTATE).
2. **trg_auditoria_delecao**: Dispara AFTER DELETE em Livros. Salva os dados apagados (OLD) e o usuário na tabela de Log.
3. **trg_limite_emprestimos**: Dispara BEFORE INSERT. Bloqueia se o aluno já tiver 3 livros emprestados.
4. **trg_preventiva_estoque**: Dispara BEFORE UPDATE. Impede que o estoque fique negativo.

3. Quatro Views (Visões)

1. **vw_acervo_publico**: Para uso do Aluno. Mostra dados do livro, ocultando preço e fornecedor.
2. **vw_livros_atrasados**: Lista empréstimos vencidos com contato do usuário.
3. **vw_ranking_leitura**: Top 10 livros mais lidos (GROUP BY/COUNT).
4. **vw_dashboard_financeiro**: Soma total de multas arrecadadas (SUM).

5. A Aplicação Java: POO, Login e Menus Inteligentes

O sistema **não** deve utilizar o terminal da IDE para funcionar, a interface deve ser construída inteiramente utilizando a biblioteca `javax.swing.JOptionPane`, caso queira usar outro tipo de interface gráfica, comunique ao professor. O objetivo não é fazer telas bonitas, mas sim **funcionais**, que respeitem as regras de segurança impostas pelo banco de dados.

O fluxo de execução obrigatório é o seguinte:

Passo 1: Conceitos de POO

O código fonte deve implementar explicitamente:

1. **Encapsulamento**:
 - o Todos os atributos das classes de modelo (Usuario, Livro, Emprestimo) devem ser private.
 - o O acesso deve ser feito obrigatoriamente via getters e setters.
 - o Validem dados nos setters (ex: não permitir preço negativo no `setPrecoCusto`).
2. **Herança**:
 - o Aproveitem a coluna tipo da tabela Usuarios para criar uma hierarquia de classes.
 - o **Classe Mãe (Superclasse)**: Criem uma classe abstrata Usuario (com id, nome, cpf, login, senha).
 - o **Classes Filhas (Subclasses)**: Criem classes concretas como Aluno e Funcionario (ou Bibliotecario) que herdem de Usuario.

3. Polimorfismo:

- o Implementem um método abstrato na classe mãe ou uma interface que se comporte de forma diferente nas subclasses.
- o *Sugestão:* Um método `getDiasPrazoEmprestimo()`.
 - Na classe `Aluno`, este método retorna **7 dias**.
 - Na classe `Funcionario`, este método retorna **14 dias**.
- o O Java deve usar esse método polimórfico para calcular a data de devolução antes de enviar para o banco.

Passo 2: Seleção de Perfil Inicial

Assim que o programa for executado, a primeira coisa que deve aparecer é uma pergunta para definir qual "caminho" o sistema vai tomar.

- **O que fazer:** Usem `JOptionPane.showOptionDialog`.
- **A Pergunta:** "Qual é o seu perfil de acesso?"
- **As Opções:** ["Funcionário", "Aluno", "Sair"].
- **Lógica:**
 - o Se escolher "Aluno" → Vai para a tela de Login e depois carrega o **Menu A**.
 - o Se escolher "Funcionário" → Vai para a tela de Login e depois carrega o **Menu B**.

Passo 3: Autenticação Real (Login no Banco)

Esta é a parte mais importante da integração. Vocês **NÃO** devem ter um usuário e senha fixos (hardcoded) no código (como `root` e `1234`).

- **O que fazer:** Abram janelas de `JOptionPane.showInputDialog` pedindo:
 1. **Usuário do Banco** (ex: `usr_estagiario`, `usr_gerente`).
 2. **Senha do Banco**.
- **A Conexão:** Passem essas variáveis digitadas para a classe `Conexao.java`.
 - o *Por que isso é vital?* Se o usuário digitar `usr_estagiario`, a conexão Java aberta terá **apenas** as permissões que vocês deram no MySQL (`REVOKE DELETE`, etc.). Se ele digitar `usr_gerente`, a conexão terá poderes totais. A segurança do Java depende de quem logou no Banco.

Passo 4: O Menu Dinâmico

Dependendo da escolha no **Passo 2**, o sistema deve exibir menus completamente diferentes.

Cenário A: Se o usuário escolheu "Aluno"

O aluno tem acesso limitado. O menu deve ser um laço (`do-while`) exibindo as seguintes opções:

1. **Consultar Acervo Disponível:**

- o *O que faz:* Executa um `SELECT * FROM vw_acervo_publico`.
 - o *Detalhe:* Mostra Título e Autor, mas **esconde** o preço e fornecedor (segurança via View).
2. **Meus Empréstimos:**
 - o *O que faz:* Executa a procedure `sp_historico_usuario` ou um `SELECT` filtrando pelo ID do aluno logado.
 3. **Sair:** Encerra a aplicação.

Cenário B: Se o usuário escolheu "Funcionário"

Aqui está o "pulo do gato". O menu deve ser **padronizado** para todos os funcionários (Gerente, Bibliotecário e Estagiário). Não tentem esconder botões via `if` no Java; deixem o banco barrar o acesso.

As opções do menu devem ser:

1. **Cadastrar Livro (Aquisição):**
 - o *O que faz:* Pede dados do livro e nota fiscal. Chama a **Transação de Aquisição** no DAO.
2. **Realizar Empréstimo:**
 - o *O que faz:* Pede ID do Usuário e do Livro. Chama a Procedure `sp_transacao_emprestimo`.
3. **Renovar Empréstimo:**
 - o *O que faz:* Pede o ID do empréstimo. Chama a Procedure `sp_renovar_emprestimo`.
4. **Realizar Devolução:**
 - o *O que faz:* Pede o ID do empréstimo. Chama a Procedure `sp_transacao_devolucao` (que calcula multa e atualiza estoque).
5. **Excluir Livro (O Teste de Fogo):**
 - o *O que faz:* Pede o ID do livro e tenta executar um `DELETE FROM livros`.
6. **Gerar Relatórios / Backup:**
 - o *O que faz:* Executa o `mysqldump` ou chama as Views de relatório financeiro.
7. **Sair.**

Passo 5: A Armadilha do Estagiário (Tratamento de Exceção)

Este é um requisito obrigatório para a avaliação da segurança. Como o menu de Funcionário é único, o **Estagiário** por exemplo verá o botão **"5. Excluir Livro"**.

1. **A Ação:** O Estagiário (logado como `usr_estagiario`) clica na opção "Excluir Livro".
2. **O Código Java:** O sistema pede o ID e chama `livroDAO.deletar(id)`.
3. **O Banco:** O MySQL recebe o comando, verifica que o usuário `usr_estagiario` possui um `REVOKE DELETE` na tabela `livros` e **bloqueia** a ação, lançando uma exceção de erro.
4. **O Resultado na Tela:**
 - o O código Java deve estar dentro de um bloco `try-catch`.
 - o No `catch (SQLException e)`, vocês devem capturar esse erro e mostrar um `JOptionPane` amigável:

"ERRO: Acesso Negado! Seu perfil de usuário não tem permissão para excluir registros do sistema."

Isso prova que a segurança da aplicação está **no banco de dados**, e não apenas "escondida" por botões desabilitados na interface.

6. Otimização e Infraestrutura

A equipe deve provar que o banco é performático utilizando **Índices** e a ferramenta **EXPLAIN**.

1. Índices (Aceleradores de Busca)

O grupo deve identificar **3 colunas** consultadas frequentemente e criar índices para elas.

- **O que fazer:** Criem índices estratégicos (ex: em título para buscas de livros ou cpf para login) e adicionem um comentário no SQL justificando a escolha.

2. Relatório EXPLAIN (A Prova Real)

Vocês devem demonstrar a melhoria de desempenho "Antes e Depois" do índice.

- **O Experimento:**
 1. Executem uma consulta pesada precedida por EXPLAIN **antes** de criar o índice. Verifiquem se a coluna type mostra **ALL** (lento).
 2. Criem o índice e rodem o EXPLAIN novamente.
 3. Comproven que a coluna type mudou para **REF** ou **CONST** (rápido).

7. Plano de Testes

A equipe deve além dos testes tradicionais (cadastrar, empréstimo, devolução, etc) deve também demonstrar estes testes de segurança na entrega:

1. **Teste de Horário:** Tentar alterar o banco às 22h (simulado). Deve dar erro de Trigger.
2. **Teste de Atomicidade (Procedure):** Usar a `sp_transacao_cadastro_completo` com um endereço inválido. Verificar que o usuário **não** foi salvo.
3. **Teste de Segurança:** Logar como Estagiário e tentar excluir um livro.
4. **Teste de Auditoria:** Excluir um livro como Gerente e checar o Log.
5. **Teste de Limite:** Tentar emprestar o 4º livro para o mesmo aluno.
6. **Teste de Renovação/Multa:**
 1. Tentar renovar um livro reservado (deve falhar).
 2. Simular uma devolução atrasada e verificar se a procedure `sp_calcular_multa` retornou o valor correto.
7. **Teste de View:** Logar como Aluno e tentar ver o "Preço de Custo" (não deve aparecer).
8. **Teste de Explain:** Rodar o EXPLAIN antes e depois de criar índice em Título.

8. Entrega e Apresentação

A avaliação final será composta pela entrega dos arquivos e pela defesa do projeto em sala de aula.

A. Apresentação em Sala

O grupo deverá apresentar o projeto utilizando **slides** contendo:

- **Capa:** Nome do projeto e integrantes.
- **Arquitetura:** Breve explicação da estrutura Java (Classes, DAO, Conexão).
- **Soluções SQL:** Destaquem a lógica das Triggers e Procedures mais complexas (não leiam código linha por linha, expliquem a lógica!).
- **Análise de Performance:** Mostrem o "Antes e Depois" do comando EXPLAIN com os índices criados.

B. Submissão no Canvas

Um integrante do grupo deve enviar um arquivo .zip contendo:

1. **Código Fonte Java:** A pasta completa do projeto (src).
2. **Script SQL de Criação:** Arquivo .sql com os comandos CREATE TABLE, CREATE USER, GRANT, CREATE PROCEDURE, CREATE TRIGGER, CREATE VIEW.
3. **Script SQL de Dados (Dump):** O arquivo de backup gerado pelo próprio sistema ou pelo mysqldump, contendo dados de teste.
4. **Slides:** O arquivo da apresentação (PDF).